

```

1  '''prmrobot.py
2
3      This is the PRM skeleton code for the robot planning problem.
4
5      PLEASE FINISH WRITING THE CODE, especially where marked by FIXME.
6
7  '''
8
9  import bisect
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import random
13 import time
14
15 from math          import inf, pi, sin, cos, sqrt, ceil, dist
16
17 from shapely.geometry import Point, LineString, Polygon
18 from shapely.geometry import MultiLineString, MultiPolygon
19 from shapely.prepared import prep
20
21 from vandercorput   import vandercorput
22
23
24 #####
25 #
26 #   Parameters
27 #
28 #   FIXME: Define the N/K...
29 #
30 N = 20
31 K = 2
32
33 # Decide whether to allow unlimited joints (360deg is same as 0deg)
34 UNLIMITEDJOINTS = True
35
36 # Define the robot dimensions
37 L1 = 1.0          # Base link
38 L2 = 1.0          # Middle link
39 L3 = 1.0          # Tip link
40
41 # Goal is  $0.5 * \sqrt{2} = 0.35$ , so distance to wall has to be  $< 0.35$ 
42 DISTANCETOWALL = 0.15
43 STEPSIZE       = 0.05
44

```

```

45 #####
46 #
47 #
48 #   Angle Wrapping Utilities (No Fixes Needed)
49 #
50
51 # Angle Wrap Utility. Return the angle wrapped into +/- 1/2 of full range.
52 def wrap(angle, fullrange):
53     return angle - fullrange * round(angle/fullrange)
54
55 def wrap180(angle): return wrap(angle, pi)
56 def wrap360(angle): return wrap(angle, 2*pi)
57
58 #####
59 #
60 #
61 #   World Definitions (No Fixes Needed)
62 #
63 #   List of obstacles/objects as well as the start/goal.
64 #
65 (xmin, xmax) = (-3.5, 3.5)
66 (ymin, ymax) = (-3.5, 3.5)
67
68 xlabel = (-3.5, -2.5, -1.5, -0.5, 0.5, 1.5, 2.5, 3.5)
69 ylabel = (-3.5, -2.5, -1.5, -0.5, 0.5, 1.5, 2.5, 3.5)
70
71 # Collect all the walls
72 walls = MultiLineString([[[ 0.5, 3.5], [0.5, 1.5], [1.0, 1.0], [3.5, 1.0]],
73                          [[ 3.5, -1.0], [1.5, -1.0], [3.5, -3.0]],
74                          [[-3.5, 1.0], [1.5, -3.5]]])
75
76 # Define the start/goal states (joint values)
77 (q1start, q2start, q3start) = (pi/2, 0, 0)
78 (q1goal, q2goal, q3goal) = (-pi/4, 0, 0)
79
80 # Define the drawing dimensions.
81 STEPSIZEDRAW = 0.5 # Joint step size when drawing the paths
82
83 #####
84 #
85 #
86 #   Visualization Class (No Fixes Needed)
87 #
88 #   This renders the world. In particular it provides the methods:
89 #       show(text = '')          Show the current figure
90 #       drawNode(node,          **kwargs) Draw a single node

```

```

91 #     drawEdge(node1, node2, **kwargs) Draw an edge between nodes
92 #     drawPath(path,          **kwargs) Draw a path (list of nodes)
93 #
94 # Visualization Class.
95 class Visualization:
96     def __init__(self):
97         # Clear the current, or create a new figure.
98         plt.clf()
99
100         # Create a new axes, enable the grid, and set axis limits.
101         plt.axes()
102         plt.grid(True)
103         plt.gca().axis('on')
104         plt.gca().set_xlim(xmin, xmax)
105         plt.gca().set_ylim(ymin, ymax)
106         plt.gca().set_xticks(xlabels)
107         plt.gca().set_yticks(ylabels)
108         plt.gca().set_aspect('equal')
109
110         # Show the walls.
111         for l in walls.geoms:
112             plt.plot(*l.xy, color='k', linewidth=2)
113
114         # Place joint 0 only once!
115         plt.gca().add_artist(plt.Circle((0,0),color='k',radius=0.05))
116
117         # Show.
118         self.show()
119
120     def show(self, text = ''):
121         # Show the plot.
122         plt.pause(0.001)
123         # If text is specified, print and wait for confirmation.
124         if len(text)>0:
125             input(text + ' (hit return to continue)')
126
127     def drawTip(self, node, **kwargs):
128         plt.arrow(node.xB, node.yB,
129                 0.9*(node.xC-node.xB), 0.9*(node.yC-node.yB),
130                 head_width=0.1, head_length=0.1, **kwargs)
131
132     def drawRobot(self, node, **kwargs):
133         plt.plot([0, node.xA, node.xB], [0, node.yA, node.yB], **kwargs)
134         self.drawTip(node, **kwargs)
135         kwargs['radius']=0.05
136         plt.gca().add_artist(plt.Circle((node.xA,node.yA), **kwargs))

```

```

137         plt.gca().add_artist(plt.Circle((node.xB,node.yB), **kwargs))
138
139     def drawNode(self, node, **kwargs):
140         self.drawTip(node, zorder=10, **kwargs)
141
142     def drawEdge(self, node1, node2, **kwargs):
143         plt.plot([(node1.xB + node1.xC)/2, (node2.xB + node2.xC)/2],
144                 [(node1.yB + node1.yC)/2, (node2.yB + node2.yC)/2], **kwargs)
145
146     def drawPath(self, path, **kwargs):
147         # Choose the larger step size to keep the drawings clean.
148         step = max(STEPSIZE, STEPSIZEDRAW)
149         for i in range(len(path)-1):
150             n = ceil(path[i].distance(path[i+1]) / step)
151             for j in range(n):
152                 node = path[i].intermediate(path[i+1], j/n)
153                 self.drawRobot(node, zorder=20, **kwargs)
154                 plt.pause(0.1)
155             self.drawRobot(path[-1], zorder=20, **kwargs)
156
157
158     #####
159     #
160     #   A* Planning Algorithm (No Fixes Needed)
161     #
162     def astar(nodes, start, goal):
163         # Clear the A* search tree information.
164         for node in nodes:
165             node.done      = False
166             node.seen      = False
167             node.parent    = None
168             node.creach    = 0
169             node.ctogoest  = inf
170
171         # Prepare the still empty *sorted* on-deck queue.
172         onDeck = []
173
174         # Begin with the start node on-deck.
175         start.done      = False
176         start.seen      = True
177         start.parent    = None
178         start.creach    = 0
179         start.ctogoest  = start.costToGoEst(goal)
180         bisect.insort(onDeck, start)
181
182         # Continually expand/build the search tree.

```

```

183 while True:
184     # Make sure we have something pending in the on-deck queue.
185     # Otherwise we were unable to find a path!
186     if not (len(onDeck) > 0):
187         return None
188
189     # Grab the next node (first on deck).
190     node = onDeck.pop(0)
191
192     # Mark this node as done and check if the goal is thereby done.
193     node.done = True
194     if goal.done:
195         break
196
197     # Add the neighbors to the on-deck queue (or update)
198     for neighbor in node.neighbors:
199         # Skip if already done.
200         if neighbor.done:
201             continue
202
203         # Compute the cost to reach the neighbor via this new path.
204         creach = node.creach + node.costToConnect(neighbor)
205
206         # Just add to on-deck if not yet seen (in correct order).
207         if not neighbor.seen:
208             neighbor.seen = True
209             neighbor.parent = node
210             neighbor.creach = creach
211             neighbor.ctogoest = neighbor.costToGoEst(goal)
212             bisect.insort(onDeck, neighbor)
213             continue
214
215         # Skip if the previous path to reach (cost) was same or better!
216         if neighbor.creach <= creach:
217             continue
218
219         # Update the neighbor's connection and resort the on-deck queue.
220         # Note the cost-to-go estimate does not change.
221         neighbor.parent = node
222         neighbor.creach = creach
223         onDeck.remove(neighbor)
224         bisect.insort(onDeck, neighbor)
225
226     # Build the path.
227     path = [goal]
228     while path[0].parent is not None:

```

```

229         path.insert(0, path[0].parent)
230
231     # Return the path.
232     return path
233
234
235     #####
236     #
237     # Node Definition (to be fixed - see FIXME)
238     #
239     class Node():
240         def __init__(self, q1, q2, q3):
241             # Define/remember the state (joint angles).
242             (self.q1, self.q2, self.q3) = (q1, q2, q3)
243
244             # Pre-compute the link positions.
245             self.xA = L1*cos(q1)
246             self.yA = L1*sin(q1)
247             self.xB = L1*cos(q1) + L2*cos(q1+q2)
248             self.yB = L1*sin(q1) + L2*sin(q1+q2)
249             self.xC = L1*cos(q1) + L2*cos(q1+q2) + L3*cos(q1+q2+q3)
250             self.yC = L1*sin(q1) + L2*sin(q1+q2) + L3*sin(q1+q2+q3)
251             self.links = LineString(
252                 [[0,0], [self.xA,self.yA], [self.xB,self.yB], [self.xC, self.yC]])
253
254             # Edges = set of neighbors. This needs to be filled in.
255             self.neighbors = set()
256
257             # Clear the status, connection, and costs for the A* search tree.
258             # TRUNK: done = True
259             # LEAF: done = False, seen = True
260             # AIR: done = False, seen = False
261             self.done = False
262             self.seen = False
263             self.parent = None
264             self.creach = 0 # Known/actual cost to get here
265             self.ctogoest = inf # Estimated cost to go from here
266
267             #####
268             # A* functions:
269             # Actual cost to connect to a neighbor and estimated to-go cost to
270             # a distant (goal) node.
271             def costToConnect(self, other):
272                 return self.distance(other)
273
274             def costToGoEst(self, other):

```

```

275         return self.distance(other)
276
277     # Define the "less-than" to enable sorting in A*. Use total cost estimate.
278     def __lt__(self, other):
279         return (self.creach + self.ctogoest) < (other.creach + other.ctogoest)
280
281     #####
282     # PRM functions:
283     # Compute the relative distance to another node.
284     def distance(self, other):
285         # FIXME: Compute and return the distance (used to sort).
286
287         # If unlimited joints, wrap around 360 degrees
288         # else, just find difference between q's
289         if UNLIMITEDJOINTS:
290             dist1 = wrap360(self.q1 - other.q1)
291             dist2 = wrap360(self.q2 - other.q2)
292             dist3 = wrap360(self.q3 - other.q3)
293         else:
294             dist1 = self.q1 - other.q1
295             dist2 = self.q2 - other.q2
296             dist3 = self.q3 - other.q3
297
298         return sqrt(dist1 ** 2 + dist2 ** 2 + dist3 ** 2)
299
300     # Check whether in free space.
301     def inFreespace(self):
302         return (walls.distance(self.links) > DISTANCETOWALL)
303
304     # Check the local planner - whether this connects to another node.
305     def connectsTo(self, other):
306         n = ceil(self.distance(other) / STEPSIZE)
307         for delta in vandercorput(1/n):
308             if not self.intermediate(other, delta).inFreespace():
309                 return False
310         return True
311
312     #####
313     # Utilities:
314     # In case we want to print the node.
315     def __repr__(self):
316         return "<Joints %6.1fdeg,%6.1fdeg,%6.1fdeg>" %
317             (self.q1 * 180/pi, self.q2 * 180/pi, self.q3 * 180/pi)
318
319     # Compute/create an intermediate node. This can be useful if you
320     # need to check the local planner by testing intermediate nodes.

```

```

321     # If allowing multiple joint rotations, this should wrap the angle
322     # difference to +/-180deg.
323     def intermediate(self, other, alpha):
324         # FIXME: Compute and return the distance (used to sort).
325
326         if UNLIMITEDJOINTS:
327             new_q1 = self.q1 + alpha * wrap360(other.q1 - self.q1)
328             new_q2 = self.q2 + alpha * wrap360(other.q2 - self.q2)
329             new_q3 = self.q3 + alpha * wrap360(other.q3 - self.q3)
330         else:
331             new_q1 = self.q1 + alpha * (other.q1 - self.q1)
332             new_q2 = self.q2 + alpha * (other.q2 - self.q2)
333             new_q3 = self.q3 + alpha * (other.q3 - self.q3)
334
335         return Node(new_q1, new_q2, new_q3)
336
337     #####
338     #
339     #   PRM Functions (to be fixed - see FIXME)
340     #
341     # Create the list of nodes.
342     def createNodesUniform(N):
343         # FIXME: create, similar to previous code
344
345         # Add nodes sampled uniformly across the space.
346         nodes = []
347         while len(nodes) < N:
348             node = Node(random.uniform(-pi, pi),
349                         random.uniform(-pi, pi),
350                         random.uniform(-pi, pi))
351             if node.inFreespace():
352                 nodes.append(node)
353         return nodes
354
355
356     # Connect to K neighbors (from all, testing nearest first)
357     def connectKNeighbors(nodes, K):
358         # FIXME: copy from the previous planner code
359
360         # Clear any existing neighbors, which are stored as a set.
361         for node in nodes:
362             node.neighbors = set()
363
364         # Check the neighbors.
365         for node in nodes:
366             # Examine the K nearest neighbors (ignoring node itself).

```



```

367         indicies = np.argsort(np.array([node.distance(n) for n in nodes]))
368
369         # Define total connections for each node and current neighbor
370         total_connect = 0
371         neighbor_index = 1
372
373         # Check if total connections has reached K neighbors for given node
374         while total_connect < K and neighbor_index < len(nodes):
375             neighbor = nodes[indicies[neighbor_index]]
376             # Force a undirected graph, so node is also neighbor's neighbor.
377             if (neighbor not in node.neighbors) and (node.connectsTo(neighbor)):
378                 node.neighbors.add(neighbor)
379                 neighbor.neighbors.add(node)
380                 total_connect += 1          # If connection successful,
381                                           # increase number of connections by 1
382                 neighbor_index += 1      # Move to next neighbor in list
383
384         # Compute the path cost
385         def pathCost(path):
386             # FIXME: copy from the previous planner code
387
388             cost = 0
389             for i in range(1, len(path)):
390                 cost += path[i-1].costToConnect(path[i])
391             return cost
392
393         # Post process the path
394         def postProcess(path):
395             # FIXME: copy from the previous planner code
396
397             i = 1
398             while i < (len(path) - 1):          # Don't include the start and goal nodes
399                 predecessor = path[i - 1]
400                 successor = path[i + 1]
401                 if (predecessor.connectsTo(successor)):
402                     path.pop(i)          # Pop the node, but don't increment i
403                                           # since next node will fall into its place
404                 else:
405                     i += 1
406
407             finalpath = path
408             return finalpath
409
410
411         #####
412         #

```

```

413 # Run/Main Code (No Fixes Needed)
414 #
415 def run():
416     # Report the parameters.
417     print('Running with', N, 'nodes and', K, 'neighbors.')
418
419     # Create the figure. Some computers seem to need an additional show()?
420     visual = Visualization()
421     visual.show()
422
423     # Create the start/goal nodes.
424     startnode = Node(q1start, q2start, q3start)
425     goalnode = Node(q1goal, q2goal, q3goal)
426
427     # Show the start/goal nodes.
428     visual.drawRobot(startnode, color='orange', linewidth=2)
429     visual.drawRobot(goalnode, color='purple', linewidth=2)
430     visual.show("Showing basic world")
431
432
433     # Create the list of sample points.
434     print("Sampling the nodes...")
435     tic = time.time()
436     nodes = createNodesUniform(N)
437     toc = time.time()
438     print("Sampled the nodes in %fsec." % (toc-tic))
439
440     # Show the sample nodes.
441     if True:
442         for node in nodes:
443             visual.drawNode(node, color='k', linewidth=1)
444             visual.show("Showing the nodes (last link only)")
445
446     # Add the start/goal nodes.
447     nodes.append(startnode)
448     nodes.append(goalnode)
449
450
451     # Connect to the nearest neighbors.
452     print("Connecting the nodes...")
453     tic = time.time()
454     connectKNeighbors(nodes, K)
455     toc = time.time()
456     print("Connected the nodes in %fsec." % (toc-tic))
457
458     # Show the neighbor connections.

```

```

459     if True:
460         for (i,node) in enumerate(nodes):
461             for neighbor in node.neighbors:
462                 if neighbor not in nodes[:i]:
463                     visual.drawEdge(node, neighbor, color='g', linewidth=1)
464             visual.show("Showing the full graph")
465
466
467     # Run the A* planner.
468     print("Running A*...")
469     tic = time.time()
470     path = astar(nodes, startnode, goalnode)
471     toc = time.time()
472     print("Ran A* in %fsec." % (toc-tic))
473
474     # If unable to connect, show the part explored.
475     if not path:
476         print("UNABLE TO FIND A PATH")
477         for node in nodes:
478             if node.done:
479                 visual.drawNode(node, color='r')
480         visual.show("Showing DONE nodes")
481         return
482
483     # Show the path.
484     cost = pathCost(path)
485     visual.drawPath(path, color='r', linewidth=2)
486     visual.show("Showing the raw path (cost/length %.1f)" % cost)
487
488
489     # Post process the path.
490     finalpath = postProcess(path)
491
492
493     # Unwrap: If allowing multiple revolutions, this shows the nodes
494     # past +/-180deg (to display the wrapping).
495     for i in range(1,len(finalpath)):
496         finalpath[i] = finalpath[i-1].intermediate(finalpath[i], 1.0)
497
498     # Report the path.
499     for node in finalpath:
500         print(node)
501
502
503     # Show the post-processed path.
504     cost = pathCost(finalpath)

```

```
505     visual.drawPath(finalpath, color='b', linewidth=2)
506     visual.show("Showing the post-processed path (cost/length %.1f)" % cost)
507
508
509 if __name__ == "__main__":
510     run()
```